LAB 02 REPORT

INTRODUCTION

The objective of this lab was to extend the existing stock tracking application by adding two new methods of retrieving historical stock price and volume data: web scraping using the Selenium and BeautifulSoup libraries, and CSV file importing using Python's built-in csv library. Both the console-based and graphical user interface versions of the program were updated and tested to support these new features.

CODEFILES

The `stock_class.py` module defines the core data structures used throughout the application. The `Stock` class represents a stock in the user's portfolio, storing the ticker symbol, company name, and number of shares owned. It uses Python property decorators to enforce data integrity — the symbol cannot be changed after creation, and shares can only be modified through the `buy()` and `sell()` methods rather than being set directly. Each `Stock` object also contains a `DataList`, which is a list of `DailyData` objects representing the historical price records for that stock. The `DailyData` class stores a single day's closing price, volume, and date, and similarly uses property decorators to manage access to these fields. Together, these two classes form the backbone of the data model that the console and GUI modules rely on.

The `stock_data.py` module handles all backend data operations and is shared between both the console and GUI versions of the program. It contains functions for creating and managing an SQLite database (`create_database()`), saving stock and daily data to the database (`save_stock_data()`), and loading that data back into memory (`load_stock_data()`). For Lab 2, two new functions were implemented. The `retrieve_stock_web()` function uses the Selenium library to control a Chrome browser, navigate to Yahoo Finance's historical data page for each stock in the portfolio, and extract price and volume records from the HTML table using BeautifulSoup. The dates entered by the user are first converted to Unix timestamps to construct the correct Yahoo Finance URL. Each valid row of data — identified by having exactly 7 columns — is used to create a `DailyData` object

which is then added to the corresponding stock. The `import_stock_web_csv()` function provides an alternative data retrieval method by reading a CSV file downloaded from Yahoo Finance. It opens the file, skips the header row, and creates `DailyData` objects from each record using the date, closing price, and volume columns.

The `stock_console.py` module implements the text-based console interface for the stock analyzer. It presents a main menu allowing the user to manage stocks, add daily data, view reports, display charts, and manage data storage. The Manage Stocks submenu allows users to add new stocks by entering a ticker symbol, company name, and share count, as well as buy or sell shares, delete stocks from the portfolio, and list all currently tracked stocks in a formatted table. The Add Daily Stock Data option allows manual entry of a date, closing price, and volume for a selected stock. The Show Report option displays a detailed summary for each stock including shares owned, latest price, highest and lowest prices, average price, total portfolio value, and record count. The Show Chart option prompts the user to select a stock and then calls the `display_stock_chart()` utility function to render a matplotlib line chart. For Lab 2, the `retrieve_from_web()` function was implemented to prompt the user for a date range and call `stock_data.retrieve_stock_web()`, displaying the number of records retrieved. The `import_csv()` function was also implemented, allowing the user to select a stock and provide a file path to a Yahoo Finance CSV file, which is then imported using `stock_data.import_stock_web_csv()`.

The `stock_GUI.py` module implements the graphical version of the stock analyzer using Python's Tkinter library. The interface features a left panel with a stock list, an Add Stock form, an Update Shares section with Buy and Sell buttons, and a Delete Stock button. The right panel displays the selected stock's name and share count as a heading, along with two tabs — History and Report — that update automatically when a stock is selected. A menu bar at the top provides access to File operations (Load and Save), Web operations, and Chart display. The `load()` and `save()` methods connect to the SQLite database through `stock_data.py`. The `add_stock()`, `buy_shares()`, `sell_shares()`, and `delete_stock()` methods manage the stock portfolio directly in the GUI. The `display_stock_data()` method refreshes the History tab with a formatted list of dates, closing prices, and volumes, and populates the Report tab with a statistical summary. For Lab 2, the `scrape_web_data()` method was implemented to prompt the user for a date range

using dialog boxes and call the web scraping backend, displaying an error message if the Chrome driver is not found. The `importCSV_web_data()` method was implemented to open a file dialog filtered for CSV files, import the selected file for the currently selected stock, and refresh the display tabs upon completion.

The `utilities.py` module provides shared utility functions used by both the console and GUI versions of the program. The `clear_screen()` function clears the terminal output on both Windows and Unix-based systems by detecting the operating system and running the appropriate system command. The `sortStocks()` function sorts the stock list alphabetically by ticker symbol using Python's built-in `sort()` method with a lambda key, ensuring the stock list is always displayed in a consistent order. The `sortDailyData()` function iterates through every stock in the portfolio and sorts its `DataList` chronologically from oldest to newest by date, which is important for accurate report generation and chart rendering. The `display_stock_chart()` function uses the matplotlib library to generate a line chart of a selected stock's closing price history over time. It plots the date on the x-axis and the closing price in dollars on the y-axis, adds a grid for readability, rotates the date labels for clarity, and displays the chart with a title formatted as "SYMBOL - Company Name." If no data exists for the stock or the symbol is not found, an appropriate message is printed instead. This function is called by both the console and GUI versions of the program.

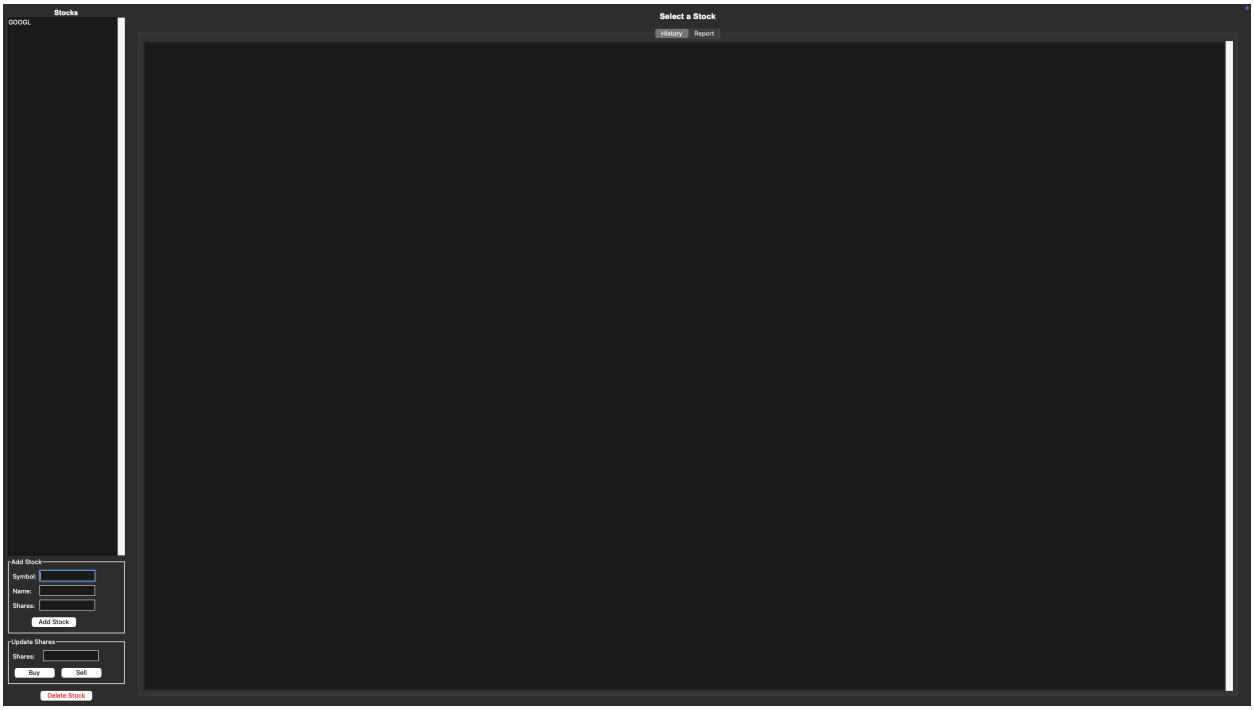
The `stocks.py` module serves as a convenient entry point for launching the application. It imports the `stock_GUI` module and calls `stock_GUI.main()` to start the graphical version of the program by default. The console version can alternatively be launched by uncommenting the `stock_console.main()` line and commenting out the GUI call, making it easy to switch between the two interfaces without modifying the core program files.

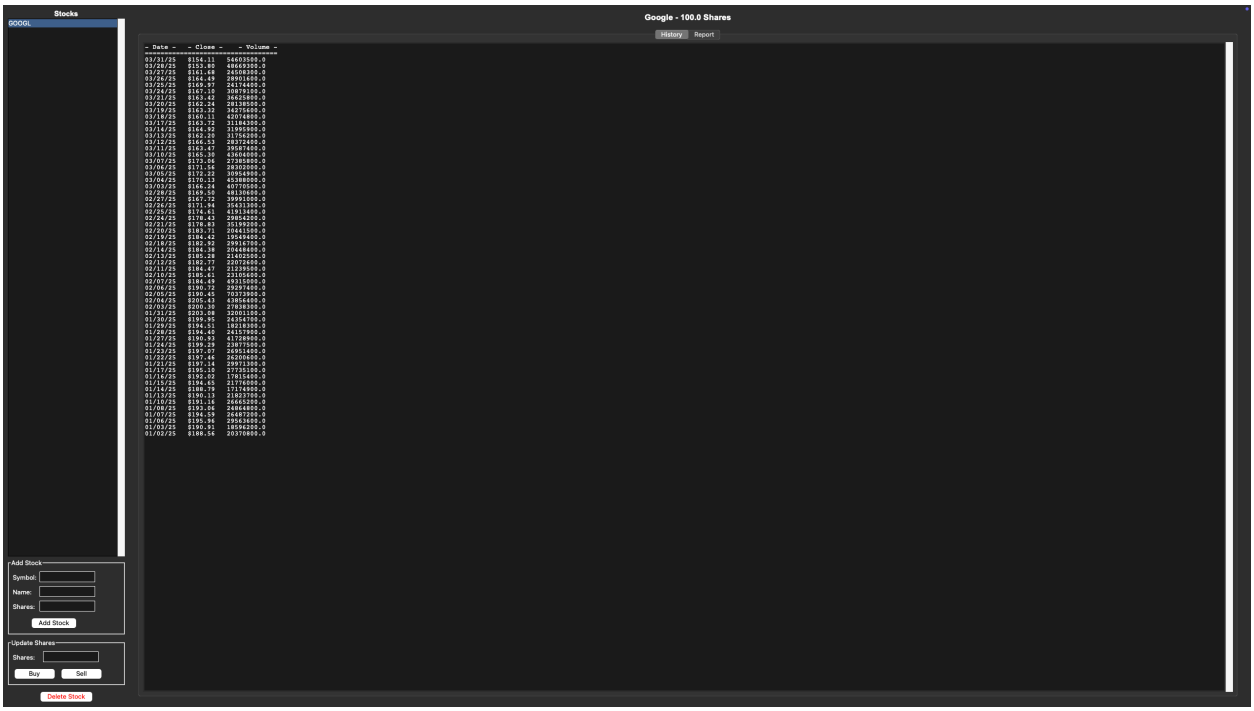
CODE TESTING

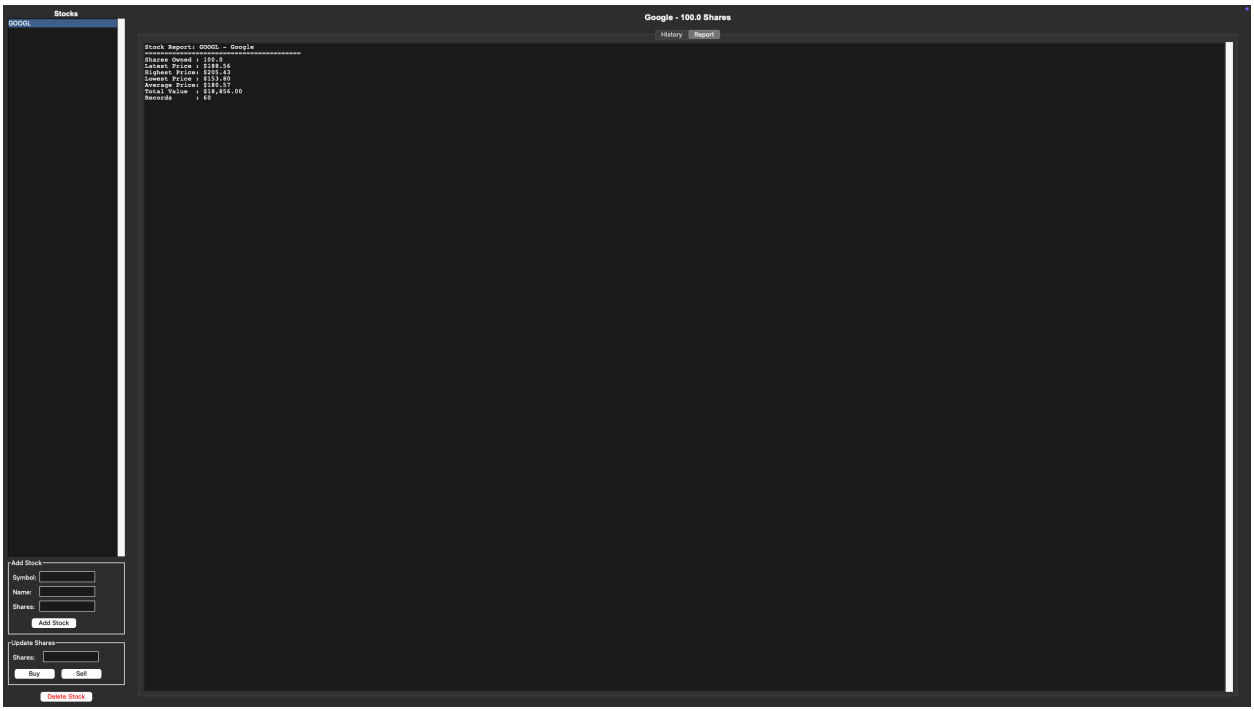
1.4

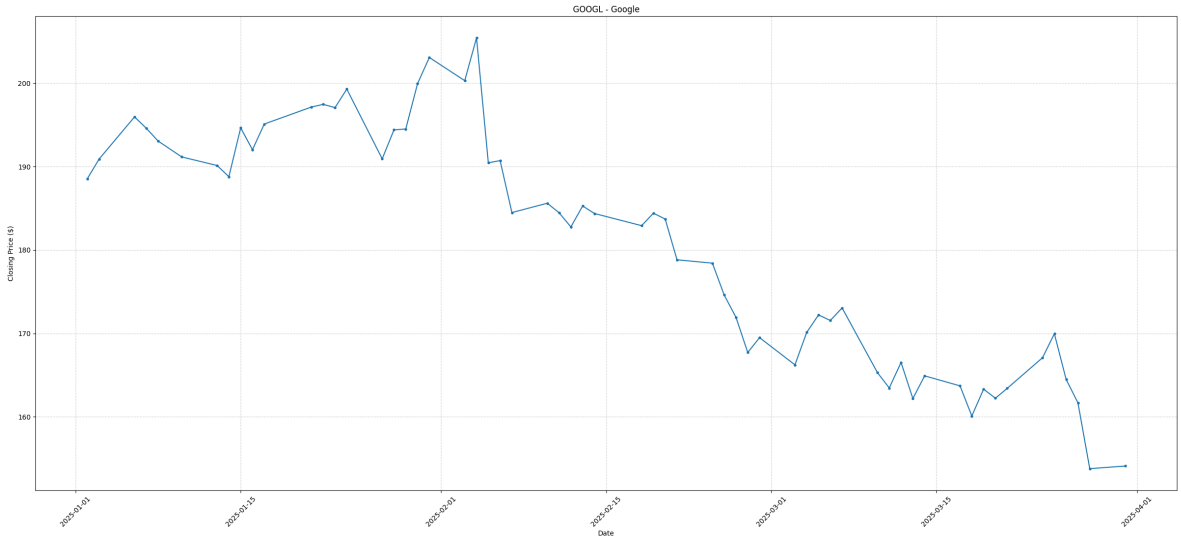
In this section, I tested the GUI version of the stock analyzer by using the Web Scraping feature to retrieve historical stock data from Yahoo! Finance. I first launched `stock_GUI.py` and added GOOGL (Google) as a stock with 100 shares. After selecting the stock, I navigated to the Web menu and selected "Scrape Data

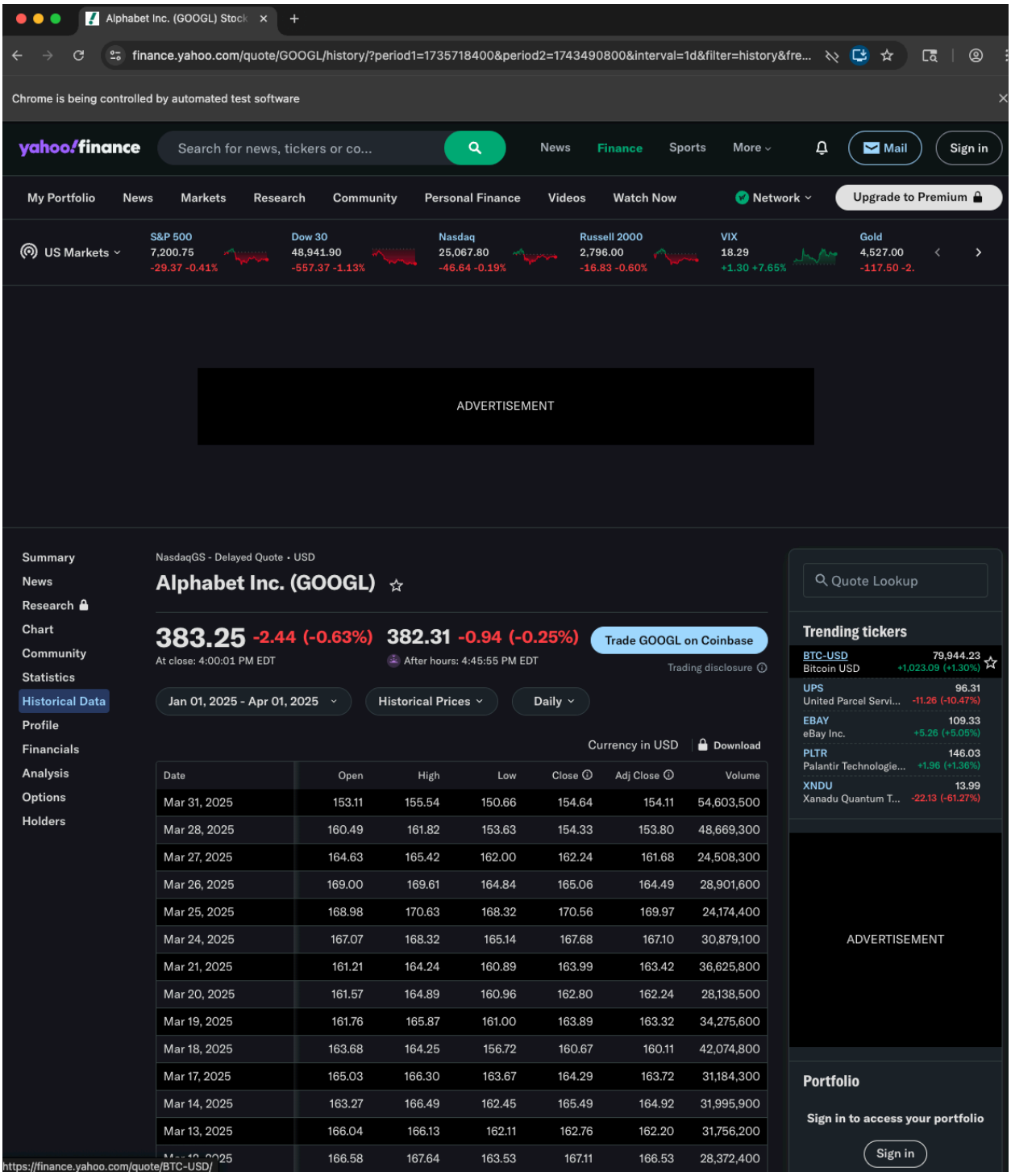
from Yahoo! Finance," which opened the Chrome browser automatically and loaded the Yahoo Finance history page for GOOGL. Once the data was retrieved, the History tab populated with date, closing price, and volume records. The Report tab then displayed a summary including latest price, highest price, lowest price, average price, and total portfolio value. Finally, I used the Chart menu to generate a line chart showing the closing price history over time, confirming the scraping functionality worked correctly.




Stocks
GOOGL
Google - 100.0 Shares
History
Report
- Date - - Close - - Volume -
03/31/25 $154.11 54603500.0
03/28/25 $153.88 48669300.0
03/27/25 $164.69 28901400.0
03/26/25 $164.40 26901400.0
03/25/25 $169.97 24174400.0
03/24/25 $167.10 30878100.0
03/21/25 $163.42 36625800.0
03/20/25 $162.24 28138500.0
03/19/25 $163.32 34275400.0
03/18/25 $165.11 42074800.0
03/17/25 $163.72 31184300.0
03/14/25 $164.92 31993800.0
03/13/25 $162.28 31794200.0
03/12/25 $161.39 37195300.0
03/11/25 $163.67 39037400.0
03/10/25 $165.30 43604000.0
03/07/25 $173.06 27385800.0
03/06/25 $171.54 28302000.0
03/05/25 $172.22 30954900.0
03/04/25 $170.13 45380800.0
03/03/25 $166.24 40770500.0
02/28/25 $169.50 48130800.0
02/27/25 $167.72 39991000.0
02/26/25 $174.84 41613300.0
02/25/25 $176.43 41613300.0
02/24/25 $178.43 28854200.0
02/21/25 $178.83 35199200.0
02/20/25 $181.71 20461500.0
02/19/25 $184.42 19549400.0
02/18/25 $183.92 29916700.0
02/14/25 $184.38 20449400.0
02/13/25 $185.28 21402500.0
02/12/25 $182.77 22072600.0
02/11/25 $184.47 21236600.0
02/10/25 $186.86 31125900.0
02/07/25 $186.49 49315800.0
02/06/25 $194.73 28287400.0
02/05/25 $190.45 70373900.0
02/04/25 $205.43 43854400.0
02/03/25 $200.38 27838300.0
01/31/25 $203.08 32001100.0
01/30/25 $199.95 24354700.0
01/29/25 $194.51 18218300.0
01/28/25 $194.40 24157900.0
01/27/25 $195.33 31307500.0
01/24/25 $197.58 43275500.0
01/23/25 $197.07 26951400.0
01/22/25 $197.46 26200600.0
01/21/25 $197.14 29971300.0
01/17/25 $195.18 27725100.0
01/16/25 $192.02 17815400.0
01/15/25 $194.65 21776000.0
01/14/25 $188.79 17174900.0
01/13/25 $189.13 21823700.0
01/10/25 $191.16 26665200.0
01/08/25 $192.70 26487300.0
01/07/25 $194.34 26487300.0
01/06/25 $195.96 29563600.0
01/03/25 $190.93 18596200.0
01/02/25 $188.56 20370800.0
Add Stock
Symbol:
Name:
Shares:
Add Stock
Update Shares
Shares:
Buy
Sell
Delete Stock


Stocks
GOOGL
Google - 100.0 Shares
History
Report
Stock Report: GOOGL - Google
Shares Owned : 100.0
Latest Price : $188.56
Highest Price: $205.43
Lowest Price : $153.80
Average Price: $174.57
Total Value : $18,856.00
Records : 60
Add Stock
Symbol:
Name:
Shares:
Add Stock
Update Shares
Shares:
Buy
Sell
Delete Stock


GOOGL - Google
Closing Price ($)
200
190
180
170
160
2025-01-01
2025-01-15
2025-02-01
2025-02-15
2025-03-01
2025-03-15
2025-04-01
Date



1.6

In this section, I tested the console version of the stock analyzer's web scraping feature by running `stock_console.py`. After loading the program, I navigated to Manage Data (option 5) and selected Load Data (option 2) to load previously

saved stocks from the database. I then selected Retrieve Data from Web (option 3) and entered a start date of 01/01/25 and an end date of 04/01/25. The program launched Chrome, navigated to Yahoo Finance for each stock in the portfolio, and successfully retrieved 60 records. I then returned to the main menu and selected Show Report (option 3), which displayed a full summary for GOOGL including shares owned, latest price, highest price, lowest price, average price, total value, and record count. The chart was also displayed successfully, showing the closing price trend over the date range.

```
Stock Analyzer ---
1 - Manage Stocks (Add, Update, Delete, List)
2 - Add Daily Stock Data (Date, Price, Volume)
3 - Show Report
4 - Show Chart
5 - Manage Data (Save, Load, Retrieve)
0 - Exit Program
Enter Menu Option: 5
```

```
Manage Data ---
1 - Save Data
2 - Load Data
3 - Retrieve Data from Web
4 - Import from CSV File
0 - Exit Manage Data
Enter Menu Option: 2
```

```
Manage Data ---
1 - Save Data
2 - Load Data
3 - Retrieve Data from Web
4 - Import from CSV File
0 - Exit Manage Data
Enter Menu Option: 3
```

```
Retrieving Stock Data from Yahoo! Finance ---
Enter Start Date (MM/DD/YY): 1/1/25
Enter End Date (MM/DD/YY): 4/1/25
Records Retrieved: 60
Press Enter to Continue
```

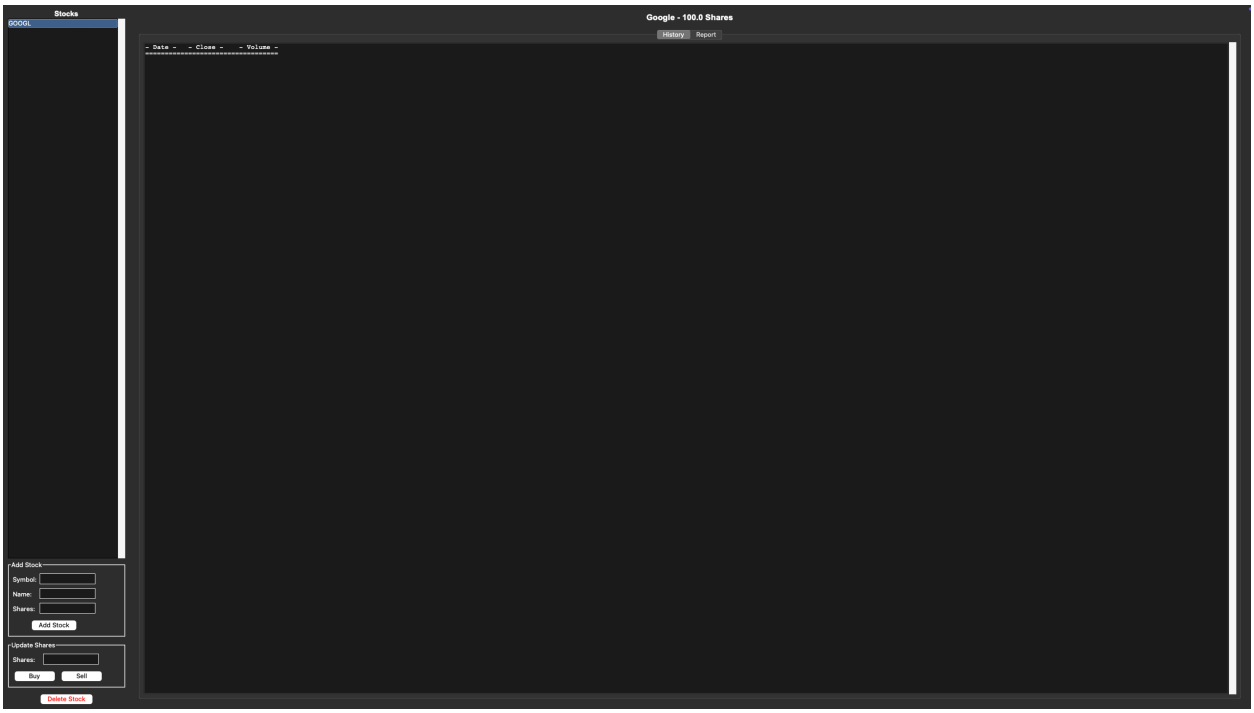
```
Stock Analyzer ---
1 - Manage Stocks (Add, Update, Delete, List)
2 - Add Daily Stock Data (Date, Price, Volume)
3 - Show Report
4 - Show Chart
5 - Manage Data (Save, Load, Retrieve)
0 - Exit Program
Enter Menu Option: 3
```

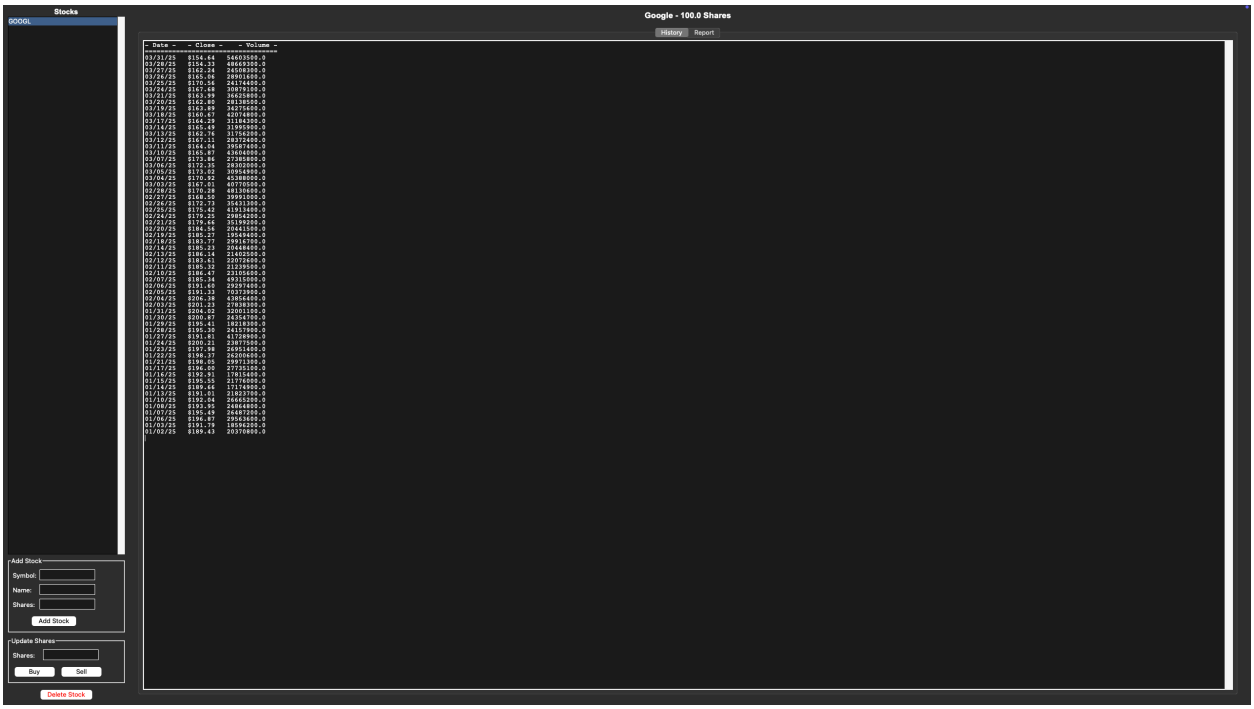
```
Stock Report ---

GOOGL - Google
=========================================
Shares Owned : 100.0
Latest Price : $154.64
Highest Price: $206.38
Lowest Price : $153.80
Average Price: $181.10
Total Value  : $15,464.00
Records      : 180

Press Enter to Continue
```

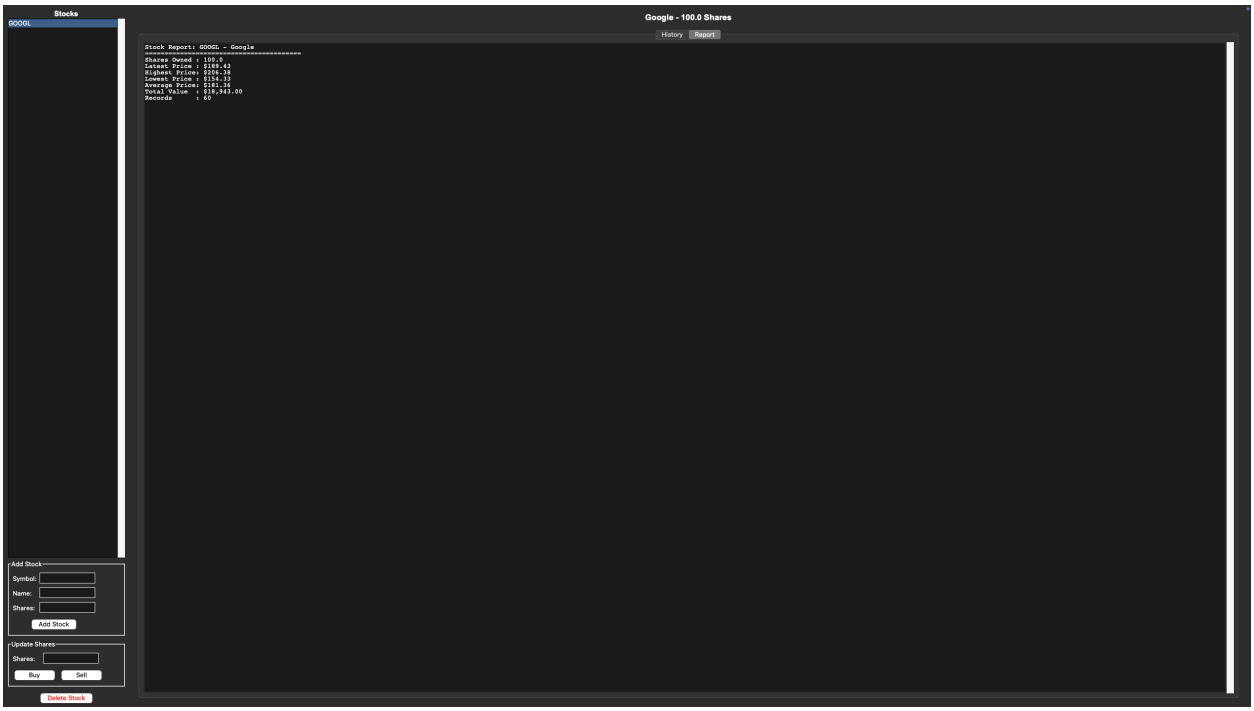
```
Display Chart ---
Stock List: [GOOGL]
Enter Stock Symbol: GOOGL
```

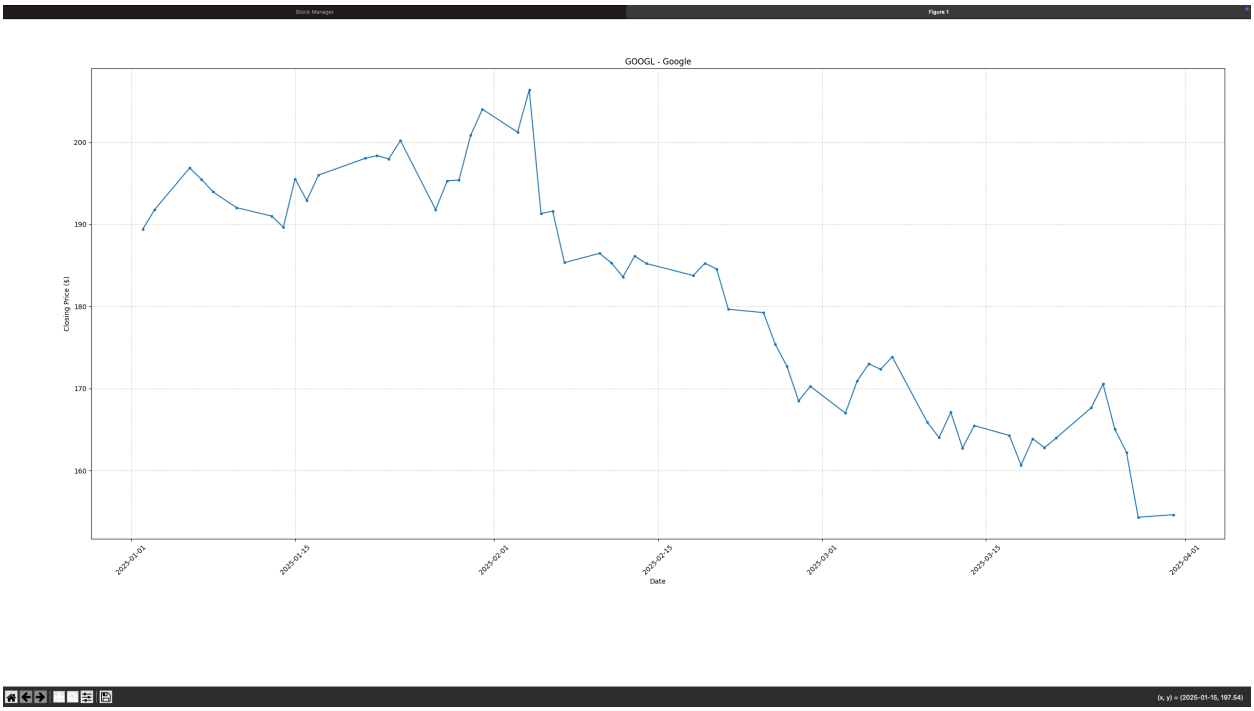
2.3

In this section, I tested the CSV import feature in the GUI version of the program. I downloaded the historical data CSV file for GOOGL from Yahoo Finance and converted the date format to match what the code expects (`YYYY-MM-DD`). I then launched `stock_GUI.py`, added GOOGL as a stock with no existing data, selected it from the stock list, and navigated to Web > Import CSV From Yahoo! Finance. I browsed to the converted CSV file and selected it. After the import completed, the History tab populated with all the date, closing price, and volume records from the CSV. The Report tab also updated to show the stock summary including shares, latest price, high, low, average, and total value, confirming the CSV import worked correctly in the GUI.


Stocks
GOOGL
Google - 100.0 Shares
History
Report
- Date - - Close - - Volume -
Add Stock
Symbol:
Name:
Shares:
Add Stock
Update Shares
Shares:
Buy
Sell
Delete Stock


Stocks
GOOGL
Google - 100.0 Shares
History
Report
- Date - - Close - - Volume -
03/31/25 $154.64 54603500.0
03/28/25 $154.33 48669300.0
03/27/25 $162.24 24509300.0
03/26/25 $165.06 28074400.0
03/25/25 $167.44 30479300.0
03/24/25 $167.48 30879300.0
03/21/25 $163.99 36629800.0
03/20/25 $162.80 28138500.0
03/19/25 $163.89 36275600.0
03/18/25 $160.67 42074800.0
03/17/25 $164.29 31184300.0
03/14/25 $165.49 31995300.0
03/13/25 $163.76 31756200.0
03/12/25 $167.11 28372400.0
03/11/25 $164.87 35604600.0
03/10/25 $164.80 27628000.0
03/07/25 $173.86 27385800.0
03/06/25 $172.35 28303000.0
03/05/25 $173.02 30954900.0
03/04/25 $170.92 45388500.0
03/03/25 $167.01 40770500.0
02/28/25 $170.28 48139600.0
02/27/25 $168.50 39991800.0
02/26/25 $172.73 31431300.0
02/25/25 $175.43 31911400.0
02/24/25 $178.15 28394300.0
02/21/25 $179.66 25194200.0
02/20/25 $184.34 20641500.0
02/19/25 $185.27 19549400.0
02/18/25 $183.77 29916700.0
02/14/25 $185.23 20448400.0
02/13/25 $186.14 21402500.0
02/12/25 $183.61 22072600.0
02/11/25 $185.32 21239500.0
02/10/25 $186.47 23155600.0
02/07/25 $191.08 19337900.0
02/06/25 $191.33 75373900.0
02/04/25 $206.38 43856400.0
02/03/25 $201.23 27838300.0
01/31/25 $204.02 32051100.0
01/30/25 $208.87 24354700.0
01/29/25 $195.41 18218300.0
01/28/25 $195.30 24157900.0
01/27/25 $191.81 41729900.0
01/24/25 $207.21 28877600.0
01/23/25 $197.96 28475400.0
01/22/25 $190.37 36300600.0
01/21/25 $194.05 29971300.0
01/17/25 $196.00 27735100.0
01/16/25 $192.91 17811400.0
01/15/25 $195.55 21776800.0
01/14/25 $189.44 17174900.0
01/13/25 $191.01 21823700.0
01/10/25 $192.04 24663200.0
01/08/25 $193.95 24864800.0
01/07/25 $192.47 26417200.0
01/06/25 $196.85 24647300.0
01/03/25 $191.79 19596300.0
01/02/25 $189.43 20370800.0
Add Stock
Symbol:
Name:
Shares:
Add Stock
Update Shares
Shares:
Buy
Sell
Delete Stock





2.5

In this section, I tested the CSV import feature in the console version of the program by running `stock_console.py`. I first added GOOGL as a new stock with 100 shares using the Manage Stocks menu (option 1 > Add Stock). I then navigated to

Manage Data (option 5) and selected Import from CSV File (option 4). When prompted, I entered the stock symbol GOOGL and provided the full file path to the downloaded CSV file. The program confirmed the import with the message "CSV File Imported." I then returned to the main menu and ran Show Report (option 3), which displayed the updated stock summary with 120 records showing the latest price, highest price, lowest price, average price, and total portfolio value. A chart was also generated displaying the full price history for GOOGL.

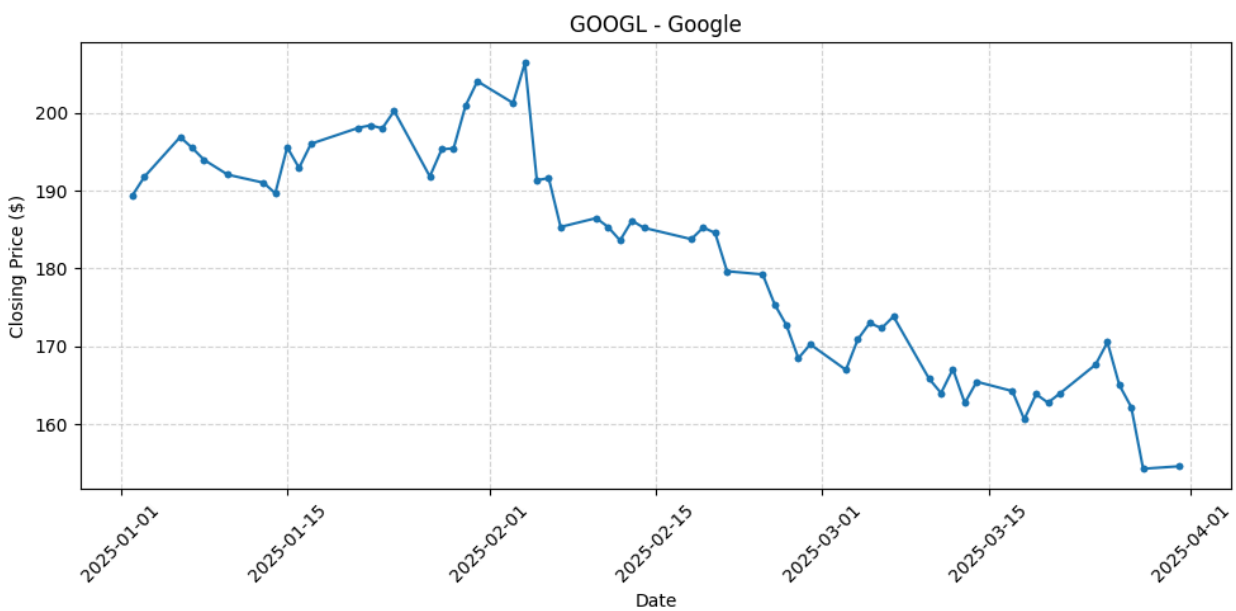
```
Add Stock ---
Enter Stock Symbol: GOOGL
Enter Stock Name: Google
Enter Number of Shares: 100
Stock GOOGL added.
Enter 0 to exit or any other key to add another stock:
```

```
Stock Analyzer ---
1 - Manage Stocks (Add, Update, Delete, List)
2 - Add Daily Stock Data (Date, Price, Volume)
3 - Show Report
4 - Show Chart
5 - Manage Data (Save, Load, Retrieve)
0 - Exit Program
Enter Menu Option: 5
```

```
Manage Data ---
1 - Save Data
2 - Load Data
3 - Retrieve Data from Web
4 - Import from CSV File
0 - Exit Manage Data
Enter Menu Option: 4
```

```
Import CSV file from Yahoo! Finance---
Stock List: [GOOGL]
Which stock do you want to use?
Enter Stock Symbol: GOOGL
Enter filename: GOOGL.csv
CSV File Imported - 60 records added.
Press Enter to Continue
```

```
Stock Report ---

GOOGL - Google
==========================================
Shares Owned : 100.0
Latest Price : $154.64
Highest Price: $206.38
Lowest Price : $154.33
Average Price: $181.36
Total Value  : $15,464.00
Records      : 120

Press Enter to Continue
```



2.6

In this section, I demonstrated the full range of console functionality as part of the sample output requirements. I added two stocks — MSFT (Microsoft) and AMZN (Amazon) — using the Add Stock feature, each with 50 shares. I then used the List Stocks option to verify both appeared correctly in the stock list. Using the Update Shares menu, I bought 100 additional shares of MSFT bringing the total to 150, and sold 20 shares of AMZN reducing its total to 30. I then demonstrated the Delete Stock feature by removing AMZN from the portfolio and confirming it no longer appeared in the list. I also tested Add Daily Stock Data by manually entering a date, closing price, and volume for MSFT, and verified it showed up correctly in the Show Report output. Finally, I used the web scraping feature to retrieve 60 days of history for MSFT and confirmed the report and chart both updated with the new data, completing all required functionality for section 2.6.

```
Add Stock ---
Enter Stock Symbol: AMZN
Enter Stock Name: Amazon
Enter Number of Shares: 50
Stock AMZN added.
Enter 0 to exit or any other key to add another stock:
```

```
Stock List ---
Symbol     Name                                    Shares
-------------------------------------------------------
MSFT       Microsoft                                50.00
AMZN       Amazon                                   50.00
Press Enter to Continue
```

```
Buy Shares ---
Stock List: [MSFT, AMZN]
Enter Stock Symbol: MSFT
Enter number of shares to buy: 100
Purchased 100.0 shares of MSFT. Total shares: 150.0
Press Enter to Continue
```

```
Sell Shares ---
Stock List: [MSFT, AMZN]
Enter Stock Symbol: AMZN
Enter number of shares to sell: 20
Sold 20.0 shares of AMZN. Total shares: 30.0
Press Enter to Continue
```

```
Delete Stock ---
Stock List: [MSFT, AMZN]
Enter Stock Symbol to Delete: AMZN
Are you sure you want to delete AMZN? (y/n): y
Stock AMZN deleted.
Press Enter to Continue
```

```
Stock List ---
Symbol     Name                                    Shares
-----------------------------------------------------------
MSFT       Microsoft                               150.00
Press Enter to Continue
```

```
Add Daily Stock Data ---
Stock List: [MSFT]
Enter Stock Symbol: MSFT
Enter Date (MM/DD/YY): 01/08/25
Enter Closing Price: 424.56
Enter Volume: 15054600
Data added for MSFT on 01/08/25.
Press Enter to Continue
```

```
Stock Report ---

MSFT - Microsoft
========================================
Shares Owned : 150.0
Latest Price : $424.56
Highest Price: $424.56
Lowest Price : $424.56
Average Price: $424.56
Total Value  : $63,684.00
Records      : 1
```

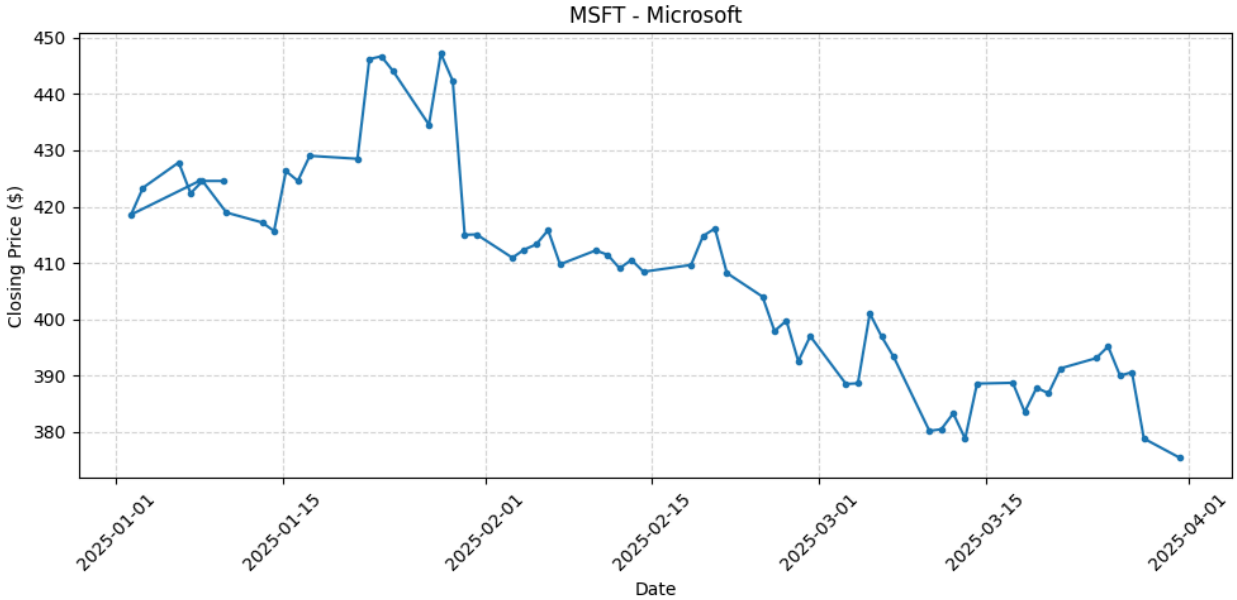
```
Stock Analyzer ---
1 - Manage Stocks (Add, Update, Delete, List)
2 - Add Daily Stock Data (Date, Price, Volume)
3 - Show Report
4 - Show Chart
5 - Manage Data (Save, Load, Retrieve)
0 - Exit Program
Enter Menu Option: 5
```

```
Manage Data ---
1 - Save Data
2 - Load Data
3 - Retrieve Data from Web
4 - Import from CSV File
0 - Exit Manage Data
Enter Menu Option: 3
```

```
Retrieving Stock Data from Yahoo! Finance ---
Enter Start Date (MM/DD/YY): 01/01/25
Enter End Date (MM/DD/YY): 04/01/25
Records Retrieved: 60
Press Enter to Continue
```

```
Stock Report ---

MSFT - Microsoft
========================================
Shares Owned : 150.0
Latest Price : $375.39
Highest Price: $447.20
Lowest Price : $375.39
Average Price: $408.25
Total Value  : $56,308.50
Records      : 62

Press Enter to Continue
```

```
Display Chart ---
Stock List: [MSFT]
Enter Stock Symbol: MSFT
```



CONCLUSION

Lab 2 successfully extended the stock tracking application with two new data retrieval methods. The web scraping implementation using Selenium and BeautifulSoup allows the program to automatically fetch historical price data directly from Yahoo Finance by controlling a real browser instance, which is necessary because the site relies heavily on JavaScript to render its data tables. The CSV import feature provides a more reliable and stable alternative by reading data files downloaded directly from Yahoo Finance, avoiding any dependency on the site's HTML structure. Both features were implemented in the shared `stock_data.py` backend and integrated into both the console and GUI interfaces. Testing confirmed that data retrieved through both methods displays correctly in the History and Report tabs of the GUI, in the console report output, and in the matplotlib stock price chart.